

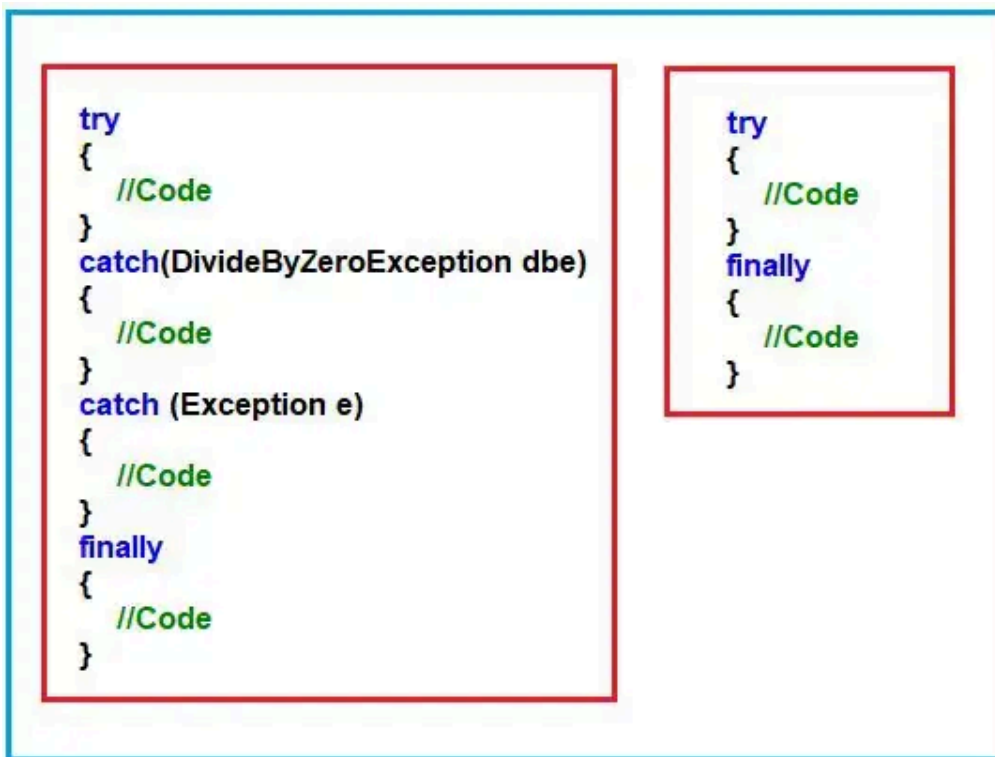
Finally Block in C# with Examples

In this article, I am going to discuss **Finally Block in C#** with examples. Please read our previous article where we discussed [Multiple Catch Blocks in C#](#) with examples. As part of this article, I am going to discuss the following pointers.

1. [What is finally block in C#?](#)
2. [Why do we need the finally block in the real-time project?](#)
3. [In how many ways we can use try-catch and finally block in C#?](#)

The Finally Block in C#

The keyword `finally` establishes a block that definitely executes the statements placed in it irrespective of whether any exception has occurred or not. That means the statements that are placed in finally block are guaranteed to be going to be executed irrespective of whether any exception is thrown or not in the try block, irrespective of whether the thrown exception is handled by the catch block or not. Following is the syntax to use finally block in C#,



As you can see, in two ways we can write the finally block in C#. They are as follows:

1. **Try, Catch, and Finally:** In this case, the exception will be handled, and stopping the abnormal termination along with the statements that are placed within the “finally” block gets executed at any cost.
2. **Try and Finally:** In this case, abnormal termination will not stop when a runtime error occurs because exceptions are not handled but even if an abnormal termination occurs, the finally blocks get executed.

Why do we need finally block in the Real-Time Project Development?

As per the industry coding standard, within the **finally** block we need to write the resource releasing logic or clean up the code. Resource releasing logic means un-referencing objects that are created in the try block. Since the statements written in the try and catch block are not guaranteed to be executed, we must place them in finally block.

For example, if we want to close ADO.NET objects such as **Connection object**, **Command object**, etc. we must call the **Close()** method in both the try as well as in the catch block to guarantee its execution. Instead of placing the same **Close()** method call statements in multiple places if we write it in the finally block it will be always executed irrespective of the exception raised or not raised.

Example to Understand the use of finally block in C#:

Let us see an example to understand the use of finally block in C#. In the below example, for the given try block, we have written two catch blocks and after the second catch block, we have written the finally block. The statements present inside the finally block is going to be executed irrespective of exception occurred or not, irrespective of the exception is handled or not. That means if we place something in the finally block, then those statements are definitely going to be executed.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number");
                Number1 = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter Second Number");
                Number2 = int.Parse(Console.ReadLine());
                Result = Number1 / Number2;
                Console.WriteLine($"Result: {Result}");
            }
            catch (DivideByZeroException DBZE)
            {
                Console.WriteLine("Second Number Should Not Be Zero");
            }
            catch (FormatException FE)
            {
                Console.WriteLine("Enter Only Integer Numbers");
            }
            finally
            {
                Console.WriteLine("Hello this is finally block...");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

Output1

```
Enter First Number
100
Enter Second Number
10
Result: 10
Hello this is finally block...
```

Output2

```
Enter First Number
100
Enter Second Number
0
Second Number Should Not Be Zero
Hello this is finally block...
```

Output3

```
Enter First Number
100
Enter Second Number
abc
Enter Only Integer Numbers
Hello this is finally block...
```

In how many ways we can use try-catch and finally block in C#?

We can use try-catch-finally in three different ways. They are as follows:

1. **Try and Catch:** In this case, the exception will be handled and stop the abnormal termination.
2. **Try, Catch, and Finally:** In this case, the exception will be handled, and stopping the abnormal termination along with the statements that are placed within the “finally” block gets executed at any cost.
3. **Try and Finally:** In this case, abnormal will not stop when a runtime error occurs because exceptions are not handled but even if an abnormal termination occurs also finally blocks get executed.

Example to Understand Try-Finally Block without Catch Block:

Let us see one example to understand the Try-Finally Block without Catch Block in C#. Please have a look at the following example. Inside the SomeMethod, we have written the logic in such a way that at runtime, this method is going to throw the Divide By Zero Exception and if you see we don't handle that Exception using the catch block. Even though the catch block is not there, the finally block is going to be executed.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void SomeMethod()
        {
```

```

    try
    {
        Console.WriteLine("Inside SomeMethod");
        int num1 = 10, num2 = 0;
        int result = num1 / num2; //Exception will be thrown here
        Console.WriteLine($"Result: {result}");
    }
    finally
    {
        Console.WriteLine("SomeMethod finally Block");
    }
}
static void Main(string[] args)
{
    try
    {
        SomeMethod();
    }
    catch (Exception)
    {
        Console.WriteLine("Exception Caught");
    }

    Console.ReadKey();
}
}
}

```

Output:

```

Inside SomeMethod
SomeMethod finally Block
Exception Caught

```

In the next article, I am going to discuss how to create a [Custom Exception in C#](#) with Examples. Here, in this article, I try to explain **Finally Block in C#** with Examples. I hope you enjoy this Finally Block in C# with Examples article.